Desafio Backend Monitoramento de rios e chuva

Um passo a passo para construir, aprender e apresentar.

A Cogmo Technology propõe este desafio a estudantes em formação. Você vai completar um pequeno serviço que ajuda uma página a encontrar dados de estações, nível de rio e chuva.

> **Sua missão**
> Criar uma API: uma porta de conversa entre a página e o servidor. A página faz um pedido, o servidor busca no banco e devolve uma resposta organizada.

Tempo sugerido: 16 a 24 horas, em até 10 dias úteis. Nível: júnior.

Antes de começar

Não é preciso conhecer hidrologia nem criar previsões. As estações e leituras são fictícias e servem apenas para praticar. Você não precisa de chave, conta ou serviço externo.

1. O que a API precisa fazer

Pense no servidor como alguém que recebe um pedido, procura a informação e devolve uma resposta. Express ajuda a receber esses pedidos; JSON é um formato de texto que organiza os campos da resposta.

Pedido da página	O que a API devolve
GET /api/stations	Lista as estações disponíveis.
GET /api/stations/EST-01/summary	Nome e leituras mais recentes.
GET /api/stations/EST-01/series?metric=level&hours=24	Histórico de nível das últimas 24 horas.
GET /api/stations/EST-01/series?metric=rain&hours=24	Histórico de chuva das últimas 24 horas.

Termos do desafio

Palavra	Explicação
Backend / servidor	Parte que recebe pedidos, consulta dados e prepara respostas.
Rota / endpoint	Endereço da API que atende a um tipo de pedido.
GET	Pedido para consultar dados, sem alterar o banco.
Banco de dados	Lugar organizado para guardar e encontrar informações.
SQLite	Um banco simples que fica em um arquivo no seu computador.

O ponto de partida

O starter já traz a estrutura, o banco e seu schema (a receita das tabelas), dados simulados e exemplos. Três funções de consulta no arquivo src/repository.js estão marcadas para completar.

2. Inicie o projeto e encontre o que precisa

Você precisa do Node.js 20 ou superior e do npm, a ferramenta que instala as bibliotecas usadas no projeto. No PowerShell, abra a pasta back e execute:

```
Copy-Item .env.example .env
npm install
npm run seed
npm run dev
```

Abra http://localhost:3000/api/health. Se aparecer status ok, o servidor está ligado. As chamadas de exemplo estão em harness/EXEMPLOS.md; o contrato completo da API está em docs/CONTRATO_API.md.

Arquivo	O que você encontrará
src/server.js	Rotas: endereços que recebem os pedidos da página.
src/repository.js	Funções que buscam informações no SQLite. Complete os TODOs.
src/db.js	A abertura do banco e a leitura do schema.
db/schema.sql	A definição das tabelas e das ligações entre elas.
scripts/seed.js	Carga local de estações e leituras inventadas.
harness/	Exemplos e um roteiro de conferência da API.

Se algo der errado

Confira se o terminal está dentro da pasta back e siga os comandos na ordem. Para encerrar o servidor, volte ao terminal e pressione Ctrl+C.

3. Como devem ser as respostas

A página precisa saber de qual estação veio o dado, quando foi medido e em que unidade. O contrato da API funciona como um combinado entre quem prepara o dado (backend) e quem o apresenta (frontend).

Exemplo de resposta resumida

```
{
  "data_mode": "simulation",
  "station": {"id": "EST-01", "name": "Estacao Aurora"},
  "latest": {
    "level_observed_at": "2026-09-28T12:00:00Z",
    "level_m": 1.42,
    "rain_observed_at": "2026-09-28T12:00:00Z",
    "rain_mm_h": 0.6
  }
}
```

level_m diz que o nível está em metros. rain_mm_h diz que a chuva está em milímetros por hora. Cada valor tem seu próprio horário. As datas terminadas em Z usam UTC, um padrão para que sistemas não confundam fusos.

Situação	Código de resposta
Estação pedida não existe	404: não foi encontrada.
Métrica ou período inválido	400: é preciso corrigir o pedido.
Algo inesperado no servidor	500: erro geral, sem mostrar detalhes internos.

> **Séries sem pontos**
> Se não houver leituras na janela, devolva uma lista vazia (points: []). Não crie valores só para preencher o gráfico.

4. Banco de dados, em linguagem simples

Imagine duas listas relacionadas. Uma descreve as estações. A outra guarda cada leitura e diz a qual estação pertence.

Tabela	O que guarda	Relação
stations	Código, nome, rio e município.	Uma estação pode ter muitas leituras.
observations	Estação, tipo de medida, horário, valor e unidade.	Cada leitura pertence a uma estação.

Chave primária é um identificador único. Chave estrangeira é o campo que aponta para uma estação. Juntas, ajudam a manter as leituras ligadas ao cadastro correto.

Ordem recomendada

Passo	Faça isso
1	Execute o seed e veja as estações que ele cria.
2	Complete listStations para listar estações ativas.
3	Complete getSummary para achar a leitura mais recente de cada tipo.
4	Complete getSeries para filtrar estação, métrica e intervalo.
5	Ordene a série do horário mais antigo para o mais recente.
6	Rode o seed de novo e confira que não criou cópias.

Uma prática simples de segurança

Valores recebidos do navegador devem ser passados ao SQL como parâmetros. Assim, o banco entende que são valores para procurar, não pedaços de um comando.

```
// O ? é preenchido com stationId pelo SQLite:
db.prepare('SELECT * FROM stations WHERE id = ?').get(stationId)
```

> **O que é um seed repetível?**
> Seed é uma carga inicial de demonstração. Repetível sem duplicar significa que você pode rodar essa carga outra vez sem criar cópias dos mesmos registros.

5. Plano de trabalho e avaliação

Etapa	Tempo	Objetivo
1. Explorar	3-4 h	Iniciar o projeto e entender as tabelas.
2. Preparar	3-5 h	Ver o seed, os exemplos e as consultas SQL.
3. Construir	5-7 h	Completar lista, resumo e histórico.
4. Conferir	3-5 h	Revisar datas, unidades, erros e resultados vazios.
5. Apresentar	2-3 h	Atualizar README, mostrar e explicar o que aprendeu.

Pronto para entregar

- Outra pessoa consegue iniciar o projeto seguindo suas instruções.
- As respostas combinam com o contrato e identificam dados simulados.
- Unidades e horários estão presentes; a série está ordenada.
- O seed pode ser repetido sem duplicar leituras.
- Erros comuns têm respostas simples e úteis.

Critérios de avaliação

Critério	Pontos	Demonstração
API e respostas	30	Mostre as três rotas e um endereço inexistente.
Banco e SQL	25	Explique as tabelas e a relação entre elas.
Cuidado com os dados	20	Mostre unidade, horário, ordem e série vazia.
Clareza do projeto	15	Demonstre que as instruções são fáceis de seguir.
Git e apresentação	10	Conte uma decisão e algo que aprendeu.

6. Glossário e conversa final

Termo	Explicação
REST	Uma forma comum de organizar endereços de uma API.
HTTP 200 / 400 / 404 / 500	Códigos que resumem sucesso, pedido inválido, item inexistente ou falha do servidor.
SQL	Linguagem para buscar e organizar dados em tabelas.
UTC / ISO 8601	Padrões que ajudam a escrever e comparar horários.
Parâmetro SQL	Valor enviado separadamente do comando para uma busca segura.
CORS	Regra que permite à página local conversar com a API local.

Na apresentação, conte

- Onde estavam as funções que você completou.
- Como uma estação se relaciona às suas leituras.
- Como você conferiu uma consulta e um erro.
- O que foi mais difícil e qual seria seu próximo pequeno passo.

Extras são opcionais

O essencial é uma API pequena que você consegue explicar. Não é preciso criar login, previsão por IA, sensores reais, publicação em nuvem ou infraestrutura avançada.

As estações e observações são inventadas para este exercício. Não representam medições de rios reais e não devem orientar alertas ou decisões.